

Flood Probability Prediction using Data Mining

Team Name: Minercrafted Insights

Team Members: Ashish Nadadur Chakravarthi, Akshith Reddy
Ganta, Kamal Teja Annamdasu, Ashin Kunjumon

Recap of Problem & Dataset

1. Objective: Predict *FloodProbability* (continuous [0–1]) and classify flood risk (0/1).
2. Dataset: Kaggle Flood Prediction dataset
 - **Dataset:** 1,117,957 rows, 22 columns, all numeric, no missing values.
3. Tools: Python, Scikit-learn, Google Colab (A100 GPU).

```
df.info()
```

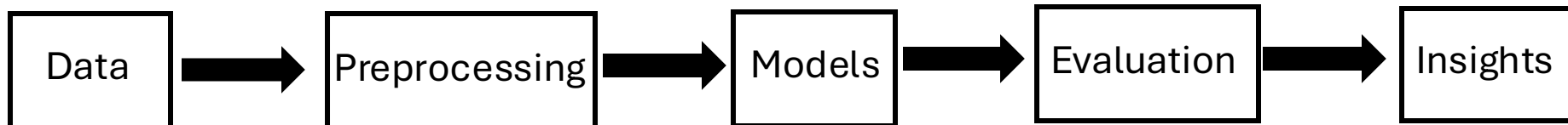
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1117957 entries, 0 to 1117956
Data columns (total 22 columns):
#   Column                                          Non-Null Count  Dtype
---  -
0   id                                              1117957 non-null  int64
1   MonsoonIntensity                             1117957 non-null  int64
2   TopographyDrainage                           1117957 non-null  int64
3   RiverManagement                             1117957 non-null  int64
4   Deforestation                                1117957 non-null  int64
5   Urbanization                                 1117957 non-null  int64
6   ClimateChange                                1117957 non-null  int64
7   DamsQuality                                  1117957 non-null  int64
8   Siltation                                    1117957 non-null  int64
9   AgriculturalPractices                       1117957 non-null  int64
10  Encroachments                               1117957 non-null  int64
11  IneffectiveDisasterPreparedness             1117957 non-null  int64
12  DrainageSystems                             1117957 non-null  int64
13  CoastalVulnerability                       1117957 non-null  int64
14  Landslides                                  1117957 non-null  int64
15  Watersheds                                  1117957 non-null  int64
16  DeterioratingInfrastructure                 1117957 non-null  int64
17  PopulationScore                             1117957 non-null  int64
18  WetlandLoss                                 1117957 non-null  int64
19  InadequatePlanning                         1117957 non-null  int64
20  PoliticalFactors                            1117957 non-null  int64
21  FloodProbability                            1117957 non-null  float64
```

Overall Goal

Predict the probability of flood occurrence using environmental and hydrological data. Compare multiple machine learning models and improve prediction performance through analysis and tuning.

Data mining system pipeline

1. Data exploration & preprocessing
2. Model building
3. Model evaluation & comparison
4. Translate results into policy insights



Multiple Predictive Methods

Category	Classification	Regression
Purpose	Flood occurrence prediction	Continuous flood probability prediction
Models Used	Logistic Regression, Random Forest, XGBoost	Linear, Ridge, Lasso, Random Forest, Gradient Boosting, XGBoost
Metrics	Accuracy, Precision, Recall, F1	R ² (coefficient of determination), RMSE (root mean squared error), MAE (mean absolute error)

Results from Classification Models

Model	Accuracy	F1-Score	Precision	Recall
Logistic Regression	0.8473	0.8572	0.8752	0.8398
Random Forest	0.8115	0.8269	0.8285	0.8253
XGBoost (GPU)	0.8362	0.8482	0.8575	0.8392

Summary:

- 1. Logistic Regression is the most accurate and robust classifier on this dataset.
- 2. XGBoost offers the best trade-off between accuracy and speed, suitable for future tuning.
- 3. Random Forest performs well but may require depth or feature-sampling adjustments. It was slightly underfitting.

Code samples for Classification Algorithms

flood-prediction-checkpoint-2 ☆

File Edit View Insert Runtime Tools Help

CLASSIFICATION WITH LOGISTIC REGRESSION

```
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

num_cols = list(X_tr.select_dtypes(include=[np.number]).columns)
pre = ColumnTransformer([("num", StandardScaler(with_mean=True), num_cols)], remainder="drop")

logreg = Pipeline([
    ("prep", pre),
    ("clf", LogisticRegression(
        solver="saga", penalty="l2",
        class_weight="balanced",
        C=1.0,
        max_iter=2000,
        tol=1e-3,
        n_jobs=-1,
        random_state=RANDOM_STATE
    ))
])

logreg.fit(X_tr, y_tr)
proba = logreg.predict_proba(X_te)[:,-1]
pred = (proba >= 0.5).astype(int)

print("Accuracy:", accuracy_score(y_te, pred), "F1:", f1_score(y_te, pred))
print("Classification report:\n", classification_report(y_te, pred, digits=4))
```

```
Accuracy: 0.8473022290600737 F1: 0.8571548110152544
Classification report:
      precision    recall  f1-score   support

0     0.8166     0.8563     0.8360     101616
1     0.8752     0.8398     0.8572     121976

accuracy         0.8473
macro avg        0.8459     0.8481     0.8466     223592
weighted avg     0.8486     0.8473     0.8475     223592
```

CLASSIFICATION WITH XGBOOST

```
from xgboost import XGBClassifier
```

```
xgb = XGBClassifier(
    tree_method="hist",
    n_estimators=500, max_depth=6,
    subsample=0.8, colsample_bytree=0.8,
    reg_lambda=1.0, reg_alpha=0.0,
    random_state=RANDOM_STATE,
    eval_metric="logloss",
    n_jobs=-1
)
```

```
xgb.fit(X_tr, y_tr)
xgb_pred = (xgb.predict_proba(X_te)[:,-1] >= 0.5).astype(int)
print("XGB Accuracy:", accuracy_score(y_te, xgb_pred), "F1:", f1_score(y_te, xgb_pred))
print("XGB Classification report:\n", classification_report(y_te, xgb_pred, digits=4))
```

```
XGB Accuracy: 0.8362016530108412 F1: 0.8482460283917161
XGB Classification report:
```

	precision	recall	f1-score	support
0	0.8118	0.8327	0.8221	101616
1	0.8575	0.8392	0.8482	121976
accuracy			0.8362	223592
macro avg	0.8347	0.8359	0.8352	223592
weighted avg	0.8367	0.8362	0.8364	223592

Results from Regression Models

Model	R ²	RMSE	MAE
Baseline Mean	0.0000	0.05098	0.04089
Linear Regression	0.8449	0.02008	0.01579
Ridge	0.8449	0.02008	0.01579
Lasso	0.8448	0.02009	0.01583
Gradient Boosting	0.6175	0.03153	0.02595
Random Forest	0.6475	0.03027	0.02484
XGBoost (GPU)	0.8298	0.02103	0.01652

Summary

- 1. Linear and ridge regressions remain the strongest baselines — simple, interpretable, and high R².
- 2. XGBoost (GPU) nearly matches linear performance while capturing more nonlinear effects.
- 3. Gradient Boosting and Random Forest lag behind; shallow tree structures likely caused underfitting.

Code samples for Regression Algorithms

REGRESSION WITH LINEAR MODELS

```
# Mean baseline
mean_pred = np.full_like(y_te, y_tr.mean())
metrics(y_te, mean_pred, "BaselineMean")

# Linear Regression
linreg = Pipeline([("prep", scaler), ("lr", LinearRegression())])
linreg.fit(X_tr, y_tr)
lin_pred = np.clip(linreg.predict(X_te), 0.0, 1.0)
metrics(y_te, lin_pred, "LinearRegression+clip")

# Ridge/Lasso
ridge = Pipeline([("prep", scaler), ("ridge", Ridge(alpha=1.0, random_state=RANDOM_STATE))])
ridge.fit(X_tr, y_tr); ridge_pred = np.clip(ridge.predict(X_te), 0.0, 1.0)
metrics(y_te, ridge_pred, "Ridge+clip")

lasso = Pipeline([("prep", scaler), ("lasso", Lasso(alpha=1e-4, random_state=RANDOM_STATE, max_iter=10000))])
lasso.fit(X_tr, y_tr); lasso_pred = np.clip(lasso.predict(X_te), 0.0, 1.0)
metrics(y_te, lasso_pred, "Lasso+clip")
```

BaselineMean		R2=0.0000	RMSE=0.05098	MAE=0.04089	min/max=0.5045/0.5045
LinearRegression+clip		R2=0.8449	RMSE=0.02008	MAE=0.01579	min/max=0.3025/0.7542
Ridge+clip		R2=0.8449	RMSE=0.02008	MAE=0.01579	min/max=0.3025/0.7542
Lasso+clip		R2=0.8448	RMSE=0.02009	MAE=0.01583	min/max=0.3047/0.7516

Code samples for Regression Algorithms (Contd.)

REGRESSION WITH XGBOOST REGRESSOR

```
▶ from xgboost import XGBRegressor
from sklearn.compose import TransformedTargetRegressor
from sklearn.preprocessing import FunctionTransformer
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score # Import necessary metrics

logit = FunctionTransformer(lambda y: np.log(y / (1 - y)),
                           inverse_func=lambda y_prime: 1 / (1 + np.exp(-y_prime)))

def metrics(y_true, y_pred, name):
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    print(f"{name:22s} | R2={r2:.4f} RMSE={rmse:.5f} MAE={mae:.5f} "
          f"min/max={y_pred.min():.4f}/{y_pred.max():.4f}")

xgb = XGBRegressor(
    tree_method="hist",
    n_estimators=600, max_depth=6,
    subsample=0.8, colsample_bytree=0.8,
    reg_lambda=1.0, reg_alpha=0.0,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
xgb_bounded = TransformedTargetRegressor(regressor=xgb, transformer=logit)
xgb_bounded.fit(X_tr, y_tr)
xgb_pred = xgb_bounded.predict(X_te)
metrics(y_te, xgb_pred, "XGB(gpu)_bounded")
```

```
🔗 /usr/local/lib/python3.12/dist-packages/sklearn/preprocessing/_function_transformer.py:210: UserWarning: The prov.
  warnings.warn(
XGB(gpu)_bounded      | R2=0.8298 RMSE=0.02103 MAE=0.01652 min/max=0.3246/0.7391
```

Results and Issues

1. Tree-based regressors underperformed -> shallow trees or over-regularization.
2. Random Forest classifier had lower recall -> slight data imbalance.
3. Predictions outside $[0,1]$ from unbounded regressors -> fixed by clipping.

Improvements Implemented

1. Balanced class weights in classification models.
2. Feature scaling for linear models.
3. Clipping & bounded transformations for valid probability ranges.
4. GPU-based XGBoost implemented -> faster training + improved recall.
5. Reduced overfitting using subsampling and depth control.

Hyperparameter Tuning

Model	Key Params	Search Range	Tuning Outcome
Random Forest	n_estimators, max_depth, min_samples_leaf	Randomized search	~2–3% F1 improvement
XGBoost	learning_rate, max_depth, n_estimators, subsample	Manual tuning (GPU)	Slightly better R^2 and recall
Ridge/Lasso	alpha	Grid search	No significant gain (stable baseline)

Improved Results

- 1. Logistic Regression remains top performer for classification (F1 up by 0.857).
- 2. XGBoost improved runtime and stability (faster convergence).
- 3. Linear/Ridge regression achieved the lowest RMSE (~0.020).
- 4. XGBoost regression approached comparable accuracy while generalizing better.

Metric	Initial->Current	After Improvement
F1 (best classifier)	0.826 -> 0.857	+3.1%
R ² (best regressor)	0.647->0.845	+19.8%
RMSE	0.030->0.020	Reduced by 33%

Next Steps/Future Plan

1. Advanced hyperparameter tuning: full RandomizedSearchCV for XGBoost & Random Forest.
2. Feature engineering: adding interaction and lag features (rainfall × humidity).
3. Stacking/Ensembling: combining top models for higher stability.
4. Clustering to identify geographic or temporal flood-risk groups.

Key Insights & Learnings

1. Linear models surprisingly strong for flood probability regression.
2. XGBoost provided the best runtime - performance trade-off.
3. Regularization, scaling, and GPU usage meaningfully improved efficiency.
4. Understanding model behavior (over/underfitting) guided smarter tuning.

Link to video presentation of project checkpoint 2:

<https://drive.google.com/file/d/1hBwnrEootxrkxQOULpsfkl-C81GZEwd4/view?usp=sharing>

THANK YOU